

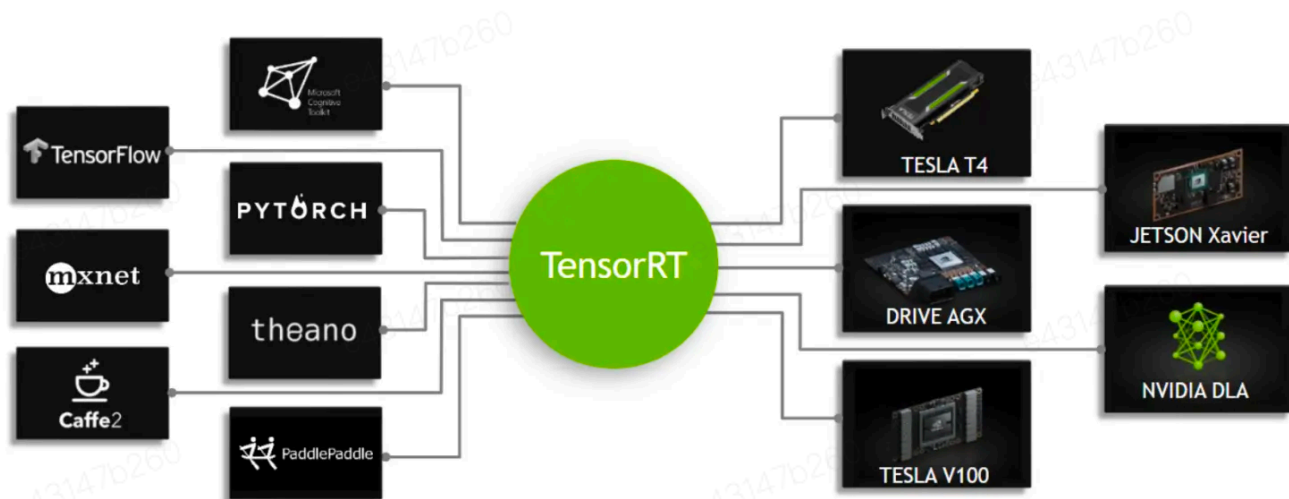
TensorRT

目录

- [TensorRT介绍](#)
- [TensorRT使用](#)
 - [1.构建流程](#)
 - [2 构建阶段](#)
 - [3 运行阶段](#)

TensorRT介绍

TensorRT 是一个高性能的深度学习推理优化器，可以为深度学习应用提供低延迟、高吞吐率的部署推理。TensorRT 可用于对超大规模数据中心、嵌入式平台或自动驾驶平台进行推理加速。TensorRT 现在已能支持 TensorFlow、Caffe、Mxnet、Pytorch 等几乎所有的深度学习框架，将 TensorRT 和 NVIDIA 的 GPU 结合起来，能在几乎所有的框架中进行快速和高效的部署推理。目前的 TensorRT 库支持 C++ 或者 Python API 部署。



加速原理

- (1) **权重与激活精度校准**：通过将模型量化为 FP16 或 INT8 来更大限度地提高吞吐量，同时保持高准确度。
- (2) **层与张量融合**：通过融合内核中的节点，优化 GPU 显存和带宽的使用。
- (3) **内核自动调整**：基于目标 GPU 平台选择最佳数据层和算法。
- (4) **动态张量显存**：更大限度减少显存占用，并高效地为张量重复利用内存。

(5) 多流执行：用于并行处理多个输入流的可扩展设计。

通过这些方法，TensorRT 能够在不损失精度的情况下显著的提升模型的推理性能。

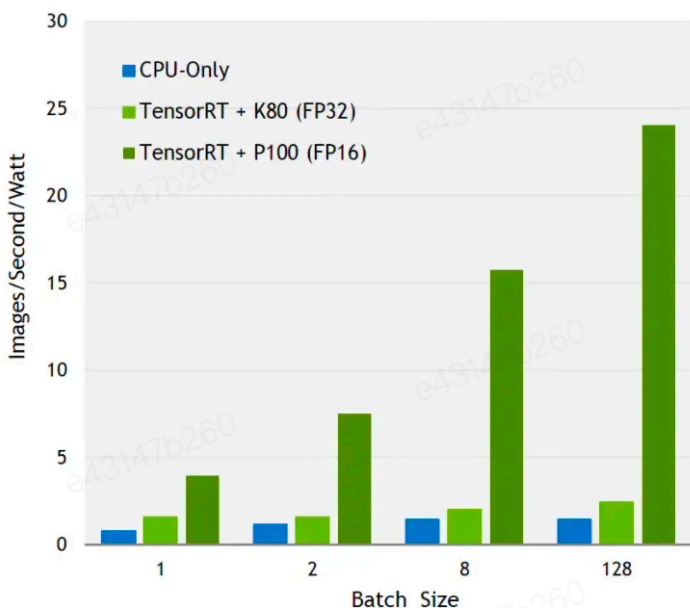
优点：

支持Batch推理：

在 GPU 上使用较大的 batch 几乎总是更有效，batch 的作用在于能尽可能多地并行计算。模型的输入只有单个 batch 的时候，单个 batch 的计算量并不能充分的利用 CUDA 核心的计算资源，有相当一部分的核心在闲置等待中；当输入有多个 batch 的时候，由于 GPU 的并行计算的特性，不同的 batch 会同步到不同的 CUDA 核心中进行并行计算，提高了单位时间 GPU 的利用率。

例如：FullyConnected 层有 V 个输入和 K 个输出，对于 1 个 batch 的实例，可以实现为 $1 \times V$ 的 input 矩阵乘以 $V \times K$ 的 weight 矩阵。如果是 N 个 batch 的实例，这就可以实现为 $N \times V$ 乘以 $V \times K$ 矩阵。将向量-矩阵乘法变为矩阵-矩阵乘法，效率更高。此外，当网络包含 MatrixMultiply 层或 FullyConnected 层时，如果硬件支持 Tensor Core，对于 FP16 和 INT8 的推理模式，将 batch 大小设置为 32 的倍数往往具有最佳性能。

Up To 16x More Energy Efficiency vs. CPU-Only Inference



GoogLeNet, Tesla P100 + TensorRT (FP16), Tesla K80 + TensorRT (FP32),
CPU-Only + Caffe (FP32)
CPU: 1 Socket Broadwell E5-2690 v4@2.6GHz with HT off

计算图合并与优化：

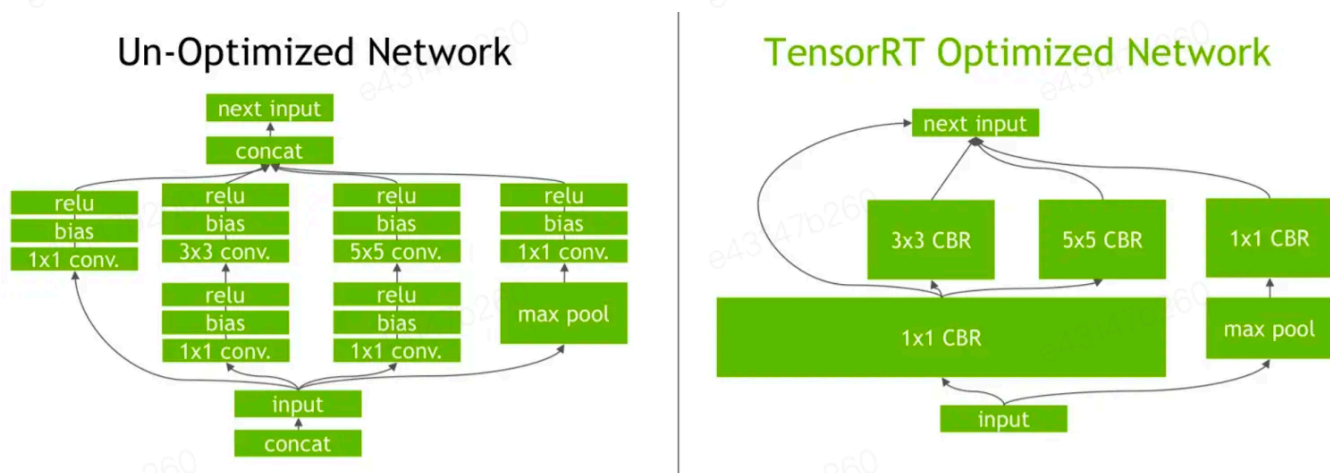
TensorRT 对于网络结构进行了重构，把一些能够合并的运算合并在了一起，针对 GPU 的特性做了优化。一个深度学习模型，在没有优化的情况下，比如一个卷积层、一个偏置层和一个激活层，这三层是需要调用三次 cuDNN 对应的 API，但实际上这三层的实现完全是可以合并到一起的，TensorRT 会对一些可以合并网络进行合并。

TensorRT 对计算图主要执行以下优化：

- 消除输出不被使用的层。
- 消除等同于无操作的操作。
- 卷积，偏置和 ReLU 操作的融合。
- 汇总具有足够相似的参数和相同的源张量的操作（例如，GoogleNet v5 的初始模块中的 1x1 卷积）。
- 通过将层输出定向到正确的最终目的地来合并串联图层。
- 在构建阶段还会在虚拟数据上运行各层，以从其核目录中选择最快的核，并在适当的情况下执行权重预格式化和内存优化。

以一个典型的 inception block 为例，优化过程如下：

1. 首先对网络结构进行垂直整合，即将目前主流神经网络的 conv、BN、Relu 三个层融合为了一个层，称之为 CBR。
2. 然后对网络结构进行水平组合，水平组合是指将输入为相同张量和执行相同操作的层融合一起。inception block 中将三个相连的 1x1 的 CBR 组合为一个大的 1x1 的 CBR。
3. 最后处理 concat 层，将 concat 层的输入直接送入下面的操作中，不用单独进行 concat 后再输入计算，相当于减少了一次传输吞吐。



量化：

深度学习模型主要是记录每个 layer（比如卷积层/全连接层）的 weights 和 bias。在 FP32 的模型中，每个 weight 数值原本需要 32-bit 的存储空间，通过 INT8 量化之后只需要 8-bit 即可。因此，模型的大小将直接降为将近 1/4。不仅模型大小明显降低，activation 采用 INT8 之后也将明显减少对内存的使用，这也意味着低精度推理过程将明显减少内存的访问带宽需求，提高高速缓存命中率，尤其对于像 batch-norm，relu，elementwise-sum 这种内存约束 (memory bound) 的 element-wise 算子来说效果更为明显。

直观的来讲，对于一个专用寄存器宽度为 512 位的 SIMD 指令，当数据类型为 FP32 而言一条指令能一次处理 16 个数值，但是当我们采用 INT8 表示数据时，一条指令一次可以处理 64 个数值。因此，在这种情况下可以让芯片的理论计算峰值增加 4 倍。

Tesla T4 GPU 引入了 Turing Tensor Core 技术，涵盖所有的精度范围，从 FP32 到 FP16 到 INT8。在 Tesla T4 GPU 上，Tensor Cores 可以进行 30 万亿次浮点计算

(TOPS)。通过 TensorRT 我们可以将一个原本为 FP32 的 weight/activation 浮点数量转化成一个 fp16/int8/uint8 的张量来处理。使用 INT8 和混合精度可以降低内存消耗，这样就跑的模型就可以更大，用于推理的 mini-batch size 可以更大，模型的单位推理速度就越快。

INT8:

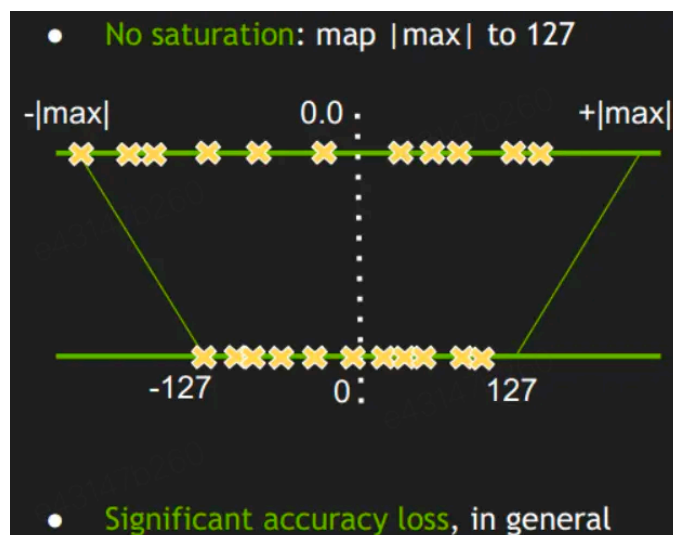
TensorRT 可以将以单精度 (FP32) 或者半精度 (FP16) 训练的模型转化为以 INT8 量化部署的模型，同时可以最小化准确率损失。由于 INT8 的表达范围远远小于 FP32，生成8位整数精度的网络时不能像 FP16 一样直接缩减精度，TensorRT 对精度为 FP32 的模型进行校验来确定中间激活的动态范围，从而确定适当的用于量化的缩放因子。

INT8 量化的本质是一种缩放 (scaling) 操作，通过缩放因子将模型的分布值从 FP32 范围缩放到 INT8 范围之内。按照量化阶段的不同，一般将量化分为 quantization aware training(QAT) 和 post-training quantization(PTQ)。QAT 需要在训练阶段就对量化误差进行建模，这种方法一般能够获得较低的精度损失。PTQ 直接对普通训练后的模型进行量化，过程简单，不需要在训练阶段考虑量化问题，因此，在实际的生产环境中对部署人员的要求也较低，但是在精度上一般要稍微逊色于 QAT。

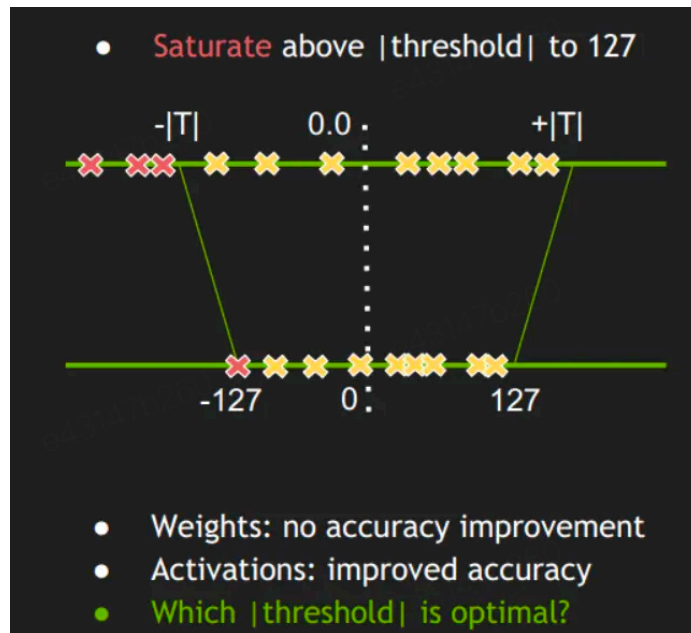
PTQ 的量化方法分为非对称算法和对称算法。非对称算法的基本思想是通过收缩因子和零点将 FP32 张量的 min/max 映射分别映射到 UINT8 数据的 min/max，min/max $\rightarrow$ 0~255。对称算法的基本思路是通过一个收缩因子将 FP32 中的最大绝对值映射到 INT8 数据的最大值，将最大绝对值的负值映射到 INT8 数据的最小值， $-|max|/|max| \rightarrow -128 \sim 127$ 。

对称算法中使用的映射方法又分为不饱和映射和饱和映射，两种映射方的区别就是 FP32 张量的值在映射后是否能够大致均匀分布在 0 的左右。如果分布不均匀，量化之后将不能够充分利用 INT8 的数据表示能力。

简单的将一个 tensor 中的 $-|max|$ 和 $|max|$ 的 FP32 值映射到 -127 和 127，中间值按照线性关系进行映射。这种对称映射关系为不饱和的 (No saturation)



根据 `tensor` 的分布计算一个阈值 `|T|`，将范围在 $\pm|T|$ 的 FP32 值映射到 ± 127 的范围中，其中 $|T| < |max|$ 。超出阈值 $\pm|T|$ 的值直接映射为 ± 127 。这种不对称的映射关系为饱和的（Saturate）。



根据实验，`weights` 在这两种方式上没有显著的差异，而对 `activation` 使用饱和的量化方式会有比较显著的性能提升。因此 `TensorRT` 在模型上使用了这两种方法进行了混合量化。

TensorRT使用

量化过程：

1. 使用PyTorch训练模型，并将其保存为PyTorch模型文件。
2. 使用TensorRT的Graphsurgeon工具将PyTorch模型转换为TensorRT可接受的ONNX格式。

TensorRT的Graphsurgeon是一个用于TensorRT模型优化的图操作工具。它是TensorRT的一个子模块，用于在构建TensorRT引擎之前对模型图进行修改和优化。

Graphsurgeon允许您通过Python代码对模型图进行操作，以满足TensorRT引擎的要求。

Graphsurgeon的主要功能包括：

- i. **修改网络结构**：可以添加、删除或修改模型中的节点，以便满足TensorRT引擎的输入要求和优化需求。
- ii. **裁剪张量**：可以裁剪张量的维度和形状，以适应不同的输入尺寸或推理需求。
- iii. **设置张量数据类型**：可以将张量的数据类型转换为TensorRT支持的数据类型，如FP32、FP16、INT8等。
- iv. **处理常量节点**：可以对常量节点进行处理，以便在TensorRT引擎中优化。
- v. **图优化**：可以对图进行优化，合并节点、融合操作等，以提高推理性能。

通过使用Graphsurgeon，您可以在TensorRT构建引擎之前对模型进行各种灵活的修改和优化，以实现更高效的推理性能。

要使用Graphsurgeon，您需要在Python脚本中导入 `import graphsurgeon as gs` 并使用Graphsurgeon API对模型进行操作。详细的使用方法和示例可以在TensorRT的官方文档中找到。

3. 使用TensorRT的Calibrator工具对模型进行量化，生成INT8量化参数。

TensorRT的Calibrator是用于INT8量化推理的关键组件之一。在INT8量化推理中，模型的权重和激活将被量化为8位整数，以减少模型的存储和计算开销，从而在NVIDIA GPU上实现更高效的推理。

Calibrator用于生成INT8量化所需的量化标定表（Calibration Table）。在INT8量化推理中，由于模型的权重和激活被量化，需要对一部分数据进行标定，以确定量化的缩放因子，使得量化后的数据尽量接近原始的FP32数据。这个过程称为量化标定。

Calibrator会使用一个小部分数据集（通常是用于标定的数据集的子集）来执行标定操作。在标定过程中，TensorRT会对模型进行推理，并比较量化后的结果与原始FP32结果之间的差异，从而计算出量化所需的缩放因子。这些缩放因子将用于量化整个模型，从而实现INT8量化推理。

为了使用Calibrator，您需要在TensorRT构建引擎时指定一个实现了 `nvInfer1::IInt8Calibrator` 接口的Calibrator对象，并将其传递给 `builder.setInt8Calibrator()` 方法。Calibrator需要实现几个方法，包括提供数据、执行推理和返回量化标定表等操作。

需要注意的是，量化标定过程需要在推理前进行，并且Calibrator需要能够在硬件上执行推理。因此，通常情况下，需要在部署环境的NVIDIA GPU上进行标定操作。

4. 使用TensorRT的Builder工具将量化参数和ONNX模型合并为TensorRT引擎文件。

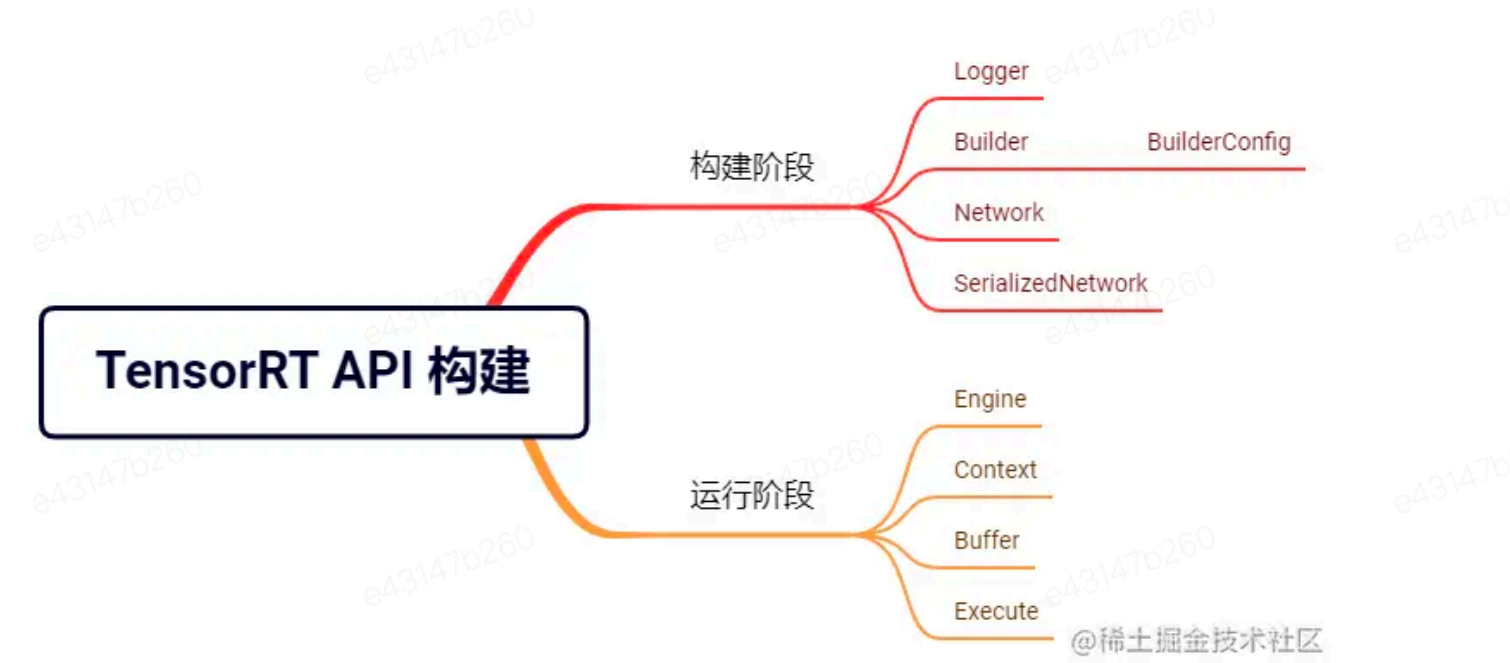
5. 在NVIDIA GPU上使用TensorRT引擎文件进行推理。

需要注意的是，PyTorch TensorRT Quantization需要使用NVIDIA GPU和安装了TensorRT和PyTorch等库的环境。此外，量化和优化过程可能需要花费一定的时间和计算资源。

模型的部署过程：

1. 构建流程

TensorRT API 的整个构建过程可以分为 **构建阶段** 和 **运行阶段**，其中构建阶段指的是添加算子&数据、网络参数配置、进行算子间逻辑连接以组建模型网，来生成 TensorRT Engine；运行阶段则可以抛开算子实现，直接调用构建阶段生成的 TensorRT Engine 进行前向推理计算。两个阶段中都有有一些比较关键的模块，在下面的图中予以列出：



2 构建阶段

2.1 构建 Logger

首先是构建 Logger 日志记录器：

</> Plain Text | 收起 ^

```
1 logger = trt.Logger(trt.Logger.VERBOSE)
```

可选参数：VERBOSE、INFO、WARNING、ERROR、INTERNAL_ERROR，产生不同等级的日志，由详细到简略：

- VERIOSE：[TensorRT] VERBOSE： Graph construction and optimization completed in 0.000261295 seconds.
- INFO： [TensorRT] INFO： Detected 1 inputs and 1 output network tensors.
- WARNING： [TensorRT] WARNING： Tensor DataType is determined at build time for tensors not marked as input or output.
- ERROR： [TensorRT] ERROR： INVALID_CONFIG： Deserialize the cuda engine failed.
- INTERNAL_ERROR： [TensorRT] ERROR： ../builder/tacticOptimizer.cpp (1820) - TRTInternal Error in computeCosts： 0 (Could not find any implementation for node (Unnamed Layer* 0) [TopK].)

2.2 构建 Builder

然后是构建 Builder 网络元数据，这是模型搭建的入口，网络的 TensorRT 内部表示以及可执行程序引擎，都是由该对象的成员方法生成的，来看 Builder 是怎么构建的：

</> Plain Text | 收起 ^

```
1 builder = trt.Builder(logger)
```

来看 Builder 的常用成员：

- `builder.max_batch_size = 256`：用于指定最大 batch size，在 static shape 模式下使用；
- `builder.max_workspace_size = 1 << 30`：用于指定最大可用显存，单位为 byte（注意：该项即将被废弃）；
- `builder.fp16_mode = True / False`：用于开启 / 关闭 fp16 模式（注意：该项即将被废弃）；
- `builder.int8_mode = True / False`：用于开启 / 关闭 int8 模式（注意：该项即将被废弃）；
- `builder.int8_calibrator = ...`：int8 模式的校准表（注意：该项即将被废弃）；
- `builder.strict_type_constraints = True / False`：开启 / 关闭强制精度模式（注意：该项即将被废弃）；
- `builder.refittable = True / False`：开启 / 关闭 refit 模式（注意：该项即将被废弃）；

你可能会比较好奇，可以看到上面很多的配置项即将被废弃，这不是说这些配置项不能用了，而是把它们移到了 `BuilderConfig` 中进行配置。需要注意的一点是，现在 NLP 里用的比较多的 Dynamic Shape 模式必须用 `BuilderConfig` 及其相关 API，那么紧接着我们来看 `BuilderConfig`。

2.3 构建 BuilderConfig

构建 `BuilderConfig` 网络元数据的选项，该项负责设置模型的一些参数，如是否开启 fp16 模式、int8 模式等。`BuilderConfig` 是建立在 `Builder` 基础之上的：

</>

Plain Text | 收起 ^

```
1 config = builder.create_builder_config()
```

来看 `BuilderConfig` 的常用成员：

- `config.max_workspace_size = 1 << 30`：用于指定最大可用显存；
- `config.max_batch_size = ...`：用于指定最大 batch，若没有配置该成员，则默认 Explicit batch 模式；
- `config.flag = ...`：用于设置标志位，如 `1 << int(trt.BuilderFlag.FP16)` 或 `1 << int(trt.BuilderFlag.INT8)`；
- `config.int8_calibrator = ...`：int8 模式的校准表；
- 等等更多高级用法

在以上的 等等更多高级用法 中还有如 `set_tactic_sources` (限制算法实现)、`set_timing_cache` (节约构建时间)、`algorithm_selector` (精确控制节点算法) 等。

2.4 构建 Network

构建 Network 计算图，是 最为核心的一个模块。Network 是网络的主体，使用 TensorRT API 搭建模型，并且标记网络的输入输出节点，以把各个计算节点织连成网状：

</>

Plain Text

收起 ^

```
1 network = builder.create_network()
```

- 常用参数：1 << int(tensorrt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)，使用 Explicit Batch 模式；
- 常用方法：
 - network.add_input('oneTensor', trt.float32, (3, 4, 5)) 标记网络输入张量；
 - convLayer = network.add_convolution_nd(XXX) 添加各种网络层；
 - network.mark_output(convLayer.get_output(0)) 标记网络输出张量；

2.5 构建 SerializedNetwork

构建 SerializedNetwork，这是网络的 TensorRT 内部表示，这个地方的后续处理就有两种选择：(1) 可以用它生成可执行的推理引擎直接进行后续推理，这样就不用反序列化的过程；(2) 把它序列化保存为文件，方便以后重新读取和使用，这里就需要涉及到反序列化，这是工程部署常用的方式。来看看怎么构建 SerializedNetwork：

</>

Plain Text

收起 ^

```
1 engineString = builder.build_serialized_network(network, config)
```

这样就完成了构建阶段。

3 运行阶段

运行阶段相比构建阶段，过程较为简洁明了。下面来看。

3.1 构建 Engine

构建 Engine，Engine 是推理引擎，是模型计算的核心，可以理解为可执行程序的代码段。来看 Engine 是怎么构建的：

</>

Plain Text

收起 ^

```
1 engine = trt.Runtime(logger).deserialize_cuda_engine(engineString)
```

3.2 构建 Context

构建 Context，主要用于计算的 GPU 上下文，类比 cpu 上的进程概念，是执行推理引擎的主体。来看 Context 是怎么构建的：

</>

Plain Text

收起 ^

```
1 context = engine.create_execution_context()
```

3.3 构建 Buffer 相关

构建 Buffer 相关，主要涉及数据的准备，包括 Host 端和 Device 端，以及数据的拷贝，如执行推理前需要将 CPU 数据拷贝到 GPU 上，即 Host -> Device；当推理完成后，需要将结果数据从 GPU 拷出到 CPU，也即 Device -> Host。一些相关的示例代码如下：

</>

Plain Text | 收起 ^

```
1  cudart.cudaMemcpy(bufferD, bufferH, bufferSize,
   cudart.cudaMemcpyKind.cudaMemcpyHostToDevice)    # Host ->
   Devicecudart.cudaMemcpy(bufferH, bufferD, bufferSize,
   cudart.cudaMemcpyKind.cudaMemcpyDeviceToHost)    # Device -> Host
```

3.4 构建 Execute

构建 Execute，Execute 调用计算核心执行计算的过程，看代码，很简单：

</>

Plain Text | 收起 ^

```
1  context.execute_v2(bufferD)
```

以上就完成了整个的 构建 与 运行过程，下面看个示例代码。

4 整流程构建示例代码

这里以构建单算子 (Identity Layer) 网络为例，进行代码展示：

</>

Plain Text | 收起 ^

```
1  ## 构建期
2  logger = trt.logger(trt.Logger.ERROR)
3  if os.path.isfile(trtFile):
4      with open(trtFile, 'rb') as f:
5          engineString = f.read()
6  else:
7      builder = trt.Builder.create_network(1 <<
   int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH))
8      profile = builder.create_optimization_profile()
9      config = builder.create_builder_config()
10     config.max_workspace_size = 1 << 30
11
12     inputTensor = network.add_input('inputT0', trt.DataType.FLOAT, [-1,
   -1, -1])
13     profile.set_shape(inputTensor.name, [1, 1, 1], [3, 4, 5], [6, 7, 8])
14     config.add_optimization_profile(profile)
15
16     identityLayer = network.add_identity(inputTensor)
17     network.mark_output(identityLayer.get_output(0))
18
```

```
19 engineString = builder.build_serialized_network(network, config)
20 with open(trtFile, 'wb') as f:
21     f.write(engineString)
22
23 ## 运行期
24 engine = trt.Runtime(logger).deserialize_cuda_engine(engineString)
25 context = engine.create_execution_context()
26
27 dataShape = [3, 4, 5]
28 context.set_binding_shape(0, dataShape)
29 data = np.arange(np.prod(dataShape),
30                 dtype=np.float32).reshape(*dataShape)
31 bufferH = [np.ascontiguousarray(data.reshape(-1))]
32 bufferD = [cudaMalloc(bufferH[0].nbytes)[1]]
33
34 cudaMemcpy(bufferD[0], bufferH[0].ctypes.data, bufferH[0].nbytes,
35             cudaMemcpyKind.cudaMemcpyHostToDevice) # Host -> Device
36 context.execute_v2(bufferD)
37 cudaMemcpy(bufferH[i].ctypes.data, bufferD[i], bufferH[i].nbytes,
38             cudaMemcpyKind.cudaMemcpyDeviceToHost) # Device -> Host
39
40 cudaFree(bufferD[0])
```